```python
import math
import sys

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from CoolProp.CoolProp import PropsSI

# Assumptions:
# Excel files are located in the same directory as this script
# Excel data starts in row 2, volume data in column A and pressure data in B
# UNO-Volume in m^3, Pressure in kPa, Temperature in Kelvin.
# PV data is ordered from process 0 -> 1 -> 2 -> 3 -> 4 -> 0

# Constants
NUM_CYL = 4
REV_PER_CYC = 2
DIESEL_LHV_MJ_KG = 44
TOT_DISP_VOL = 0.00221670777637296 # Based on bore and stroke data for engine
ROOM_TEMP_C = 21 # deg C
ROOM_TEMP_K = ROOM_TEMP_C + 273
GAS_CONST_AIR_KJ = 0.287055

engine_rpm = None
power_break_kW = None

# Allow user selection of rpm
options = [1400, 2000]
print("Select an engine rpm:")
for i, rpm in enumerate(options, start=1):
    print(f"{i}. {rpm} rpm")

while True:
    choice = input("Enter option number: ")
    if choice in ["1", "2"]:
        engine_rpm = options[int(choice)-1]
        break
    print("Invalid choice. Try again.")

# Set the working excel document based on rpm selection
if engine_rpm == 2000:
    df = pd.read_excel('Diesel_PV_Data_2000RPM.xlsx')
    # These values are set from calculated values.
    engine_rpm_actual = 2000 # Measured same as nominal
    power_break_kW = 22.9
    m_flow_rate_air_kg_s = 0.0311979686346883
    m_flow_rate_exh_kg_s = 0.0329753019680216
    m_flow_rate_fuel_kg_s = 0.00177733333333333
elif engine_rpm == 1400:
    df = pd.read_excel('Diesel_PV_Data_1400RPM.xlsx')
    engine_rpm_actual = 1390 # Update to measured value
    power_break_kW = 17.7
    m_flow_rate_air_kg_s = 0.0217706507318485
    m_flow_rate_exh_kg_s = 0.0231753173985152
    m_flow_rate_fuel_kg_s = 0.00140466666666667
```

```
# Import the data as columns from datafrom and convert to numpy array
vol_data = df.iloc[:, 0].to_numpy()
prs_data = df.iloc[:, 1].to_numpy()

# Maximum volume corresponds to V1 and V4
max_vol = np.max(vol_data)

index_max_vol = np.where(vol_data == max_vol)[0]

# more than one value will be returned, select second value, as V4 occurs
# after V1.
V4 = vol_data[index_max_vol][1]
P4 = prs_data[index_max_vol][1]

# Sample data is for test plots
pt4_sample_data_up_idx = index_max_vol[1] + 200
pt4_sample_data_low_idx = index_max_vol[1] - 200

P4_sample_data = prs_data[pt4_sample_data_low_idx : pt4_sample_data_up_idx]
V4_sample_data = vol_data[pt4_sample_data_low_idx : pt4_sample_data_up_idx]

# Test plot below to confirm correct points obtained.
# plt.figure(1)
# plt.plot(V4, P4, 'ro')
# plt.plot(V4_sample_data,P4_sample_data)
# plt.grid(True)
# plt.title("Point 4 Verification")
# plt.xlabel("Volume [m^3]")
# plt.ylabel("Pressure [kPa]")
# plt.xticks(rotation=45)
# plt.show()

# Filter out intake and exhaust stroke data.
mask_prs_abv_P4 = prs_data > P4
trimmed_vol_data = vol_data[mask_prs_abv_P4]

min_vol = np.min(trimmed_vol_data)
min_vol2 = np.min(vol_data)

index_min_vol = np.where(vol_data == min_vol)[0]

V2 = vol_data[index_min_vol][0]
P2 = prs_data[index_min_vol][0]

pt2_sample_data_up_idx = index_min_vol[0] + 200
pt2_sample_data_low_idx = index_min_vol[0] - 200

P2_sample_data = prs_data[pt2_sample_data_low_idx : pt2_sample_data_up_idx]
V2_sample_data = vol_data[pt2_sample_data_low_idx : pt2_sample_data_up_idx]

# plt.figure(2)
# plt.plot(V2, P2, 'ro')
# plt.plot(V2_sample_data,P2_sample_data)
# plt.grid(True)
# plt.title("Point 2 Verification")
# plt.xlabel("Volume [m^3]")
```

```
# plt.ylabel("Pressure [kPa]")
# plt.xticks(rotation=45)
# plt.show()

# Both volumes correspond to volume at BDC.
V1 = V4

# Select first occurance as V1 occurs before V4
index_V1 = index_max_vol[0]

P1 = prs_data[index_V1]

pt1_sample_data_up_idx = index_V1 + 200
pt1_sample_data_low_idx = index_V1 - 200

P1_sample_data = prs_data[pt1_sample_data_low_idx : pt1_sample_data_up_idx]
V1_sample_data = vol_data[pt1_sample_data_low_idx : pt1_sample_data_up_idx]

# plt.figure(3)
# plt.plot(V1, P1, 'ro')
# plt.plot(V1_sample_data,P1_sample_data)
# plt.grid(True)
# plt.title("Point 1 Verification")
# plt.xlabel("Volume [m^3]")
# plt.ylabel("Pressure [kPa]")
# plt.xticks(rotation=45)
# plt.show()

max_prs = np.max(prs_data)

index_max_prs = np.where(prs_data == max_prs)[0][0]
pt3_sample_data_low_idx = index_max_prs

pt3_sample_data_up_idx = None
threshold_factor = 0.75

# Select a portion of data, proportional to P2 after the max pressure
for index in range(pt3_sample_data_low_idx, len(prs_data)):
    if prs_data[index] < P2 * threshold_factor:
        pt3_sample_data_up_idx = index
        break

P3_sample_data = prs_data[pt3_sample_data_low_idx : pt3_sample_data_up_idx]
V3_sample_data = vol_data[pt3_sample_data_low_idx : pt3_sample_data_up_idx]

# Interpolating from left to right
# Interpolation used to find V3 with matching P3
# Due to pressure variation there is not likely a matching P2 value before and
# after the max pressure.
# Essentially drawing a line across from (P2,V2)1 to intersection, then
# interpolating to find V3.
index_right_P3 = None
for index in range(1, len(P3_sample_data)):
    # If the value is less than P3/P2 then it will be to the right of P3.
    if P3_sample_data[index] < P2 <= P3_sample_data[index-1]:
        index_right_P3 = index
```

```
        break

index_left_P3 = index_right_P3 - 1

left_P3 = P3_sample_data[index_left_P3]
right_P3 = P3_sample_data[index_right_P3]
left_V3 = V3_sample_data[index_left_P3]
right_V3 = V3_sample_data[index_right_P3]

# Perform interpolation calculation
P3 = P2
V3 = left_V3 + ((P3 - left_P3) / (right_P3 - left_P3)) * (right_V3 - left_V3)

# plt.figure(4)
# plt.plot(V3, P3, 'ro')
# plt.plot(V3_sample_data,P3_sample_data)
# plt.grid(True)
# plt.title("Point 3 Verification")
# plt.xlabel("Volume [m^3]")
# plt.ylabel("Pressure [kPa]")
# plt.xticks(rotation=45)
# plt.show()

# Calculate the indicated work
work_i_kJ_cyl = np.trapezoid(prs_data, vol_data)

# Calculate the indicated power
power_i_kW = work_i_kJ_cyl * engine_rpm_actual * NUM_CYL / (REV_PER_CYC *60)

# Calculate mechanical efficiency
eff_mech = power_break_kW / power_i_kW
eff_mech_perc = eff_mech * 100

# Calculate indicated mean effective pressure
imep_MPa = (work_i_kJ_cyl / 1000) * NUM_CYL / TOT_DISP_VOL

# Polytropic coefficients of compression and expansion.
n_comp = math.log(P1/P2) / math.log(V2/V1)
n_exp = math.log(P3/P4) / math.log(V4/V3)

V1_ideal = V1
V2_ideal = V2
V4_ideal = V4

# Pick an arbitrary point in the middle of the compression stroke
mid_pt_comp_stroke = (
    index_max_vol[0]
    + ((index_min_vol[0] - index_max_vol[0]) / 2) # Index V1 < index V2
)

# Array index must be integer
mid_pt_comp_stroke = int(mid_pt_comp_stroke)

# Solve for P2 using polytropic relationship between V2 and P,V @ arbitrary point.
P2_ideal = (
    prs_data[mid_pt_comp_stroke]
```

```
    * vol_data[mid_pt_comp_stroke]**n_comp
    / V2_ideal**n_comp
)

# Solve for P1 using polytropic relationship between P1,V1 and P2,V2 (ideal)
P1_ideal = P2_ideal * (V2_ideal / V1_ideal)**n_comp

# Pick an arbitrary point in the middle of the expansion stroke
mid_pt_exp_stroke = index_max_prs + ((index_max_vol[1] - index_max_prs) / 2)

# Array index must be integer
mid_pt_exp_stroke = int(mid_pt_exp_stroke)

# Solve for V3 using polytropic relationsip between P3 and P,V @ arbitrary point.
P3_ideal = P2_ideal
V3_ideal = (
    (prs_data[mid_pt_exp_stroke] / P3_ideal)**(1 / n_exp)
    * vol_data[mid_pt_exp_stroke]
)

# Solve for remaining P4 ideal
P4_ideal = P3_ideal * (V3_ideal / V4_ideal)**n_exp

# Generate curve data between ideal points
vol_data_ideal_comp = np.linspace(V2_ideal, V1_ideal, 1000)
prs_data_ideal_comp = P1_ideal * (V1_ideal / vol_data_ideal_comp)**n_comp

vol_data_ideal_pconst = np.linspace(V2_ideal, V3_ideal, 100)
prs_data_ideal_pconst = np.full_like(vol_data_ideal_pconst, P3_ideal)

vol_data_ideal_exp = np.linspace(V3_ideal, V4_ideal, 1000)
prs_data_ideal_exp = P3_ideal * (V3_ideal / vol_data_ideal_exp)**n_exp

prs_data_ideal_vconst = np.linspace(P1_ideal, P4_ideal, 100)
vol_data_ideal_vconst = np.full_like(prs_data_ideal_vconst, V1_ideal)

# Calculate cut-off ratio
cut_off_ratio = V3_ideal / V2_ideal

# Calculate the mass of the air in the cylinder
mass_air_cyl = m_flow_rate_air_kg_s * REV_PER_CYC / engine_rpm_actual * 60 / NUM_CYL

# Calculate T2 using ideal gas law state equation
T2_ideal = P2_ideal * V2_ideal / mass_air_cyl / GAS_CONST_AIR_KJ

# Calculate T3 using constant pressure assumption
T3_ideal = V3_ideal * T2_ideal / V2_ideal

# The following method may not be the correct one.
# # Using air standard assumptions, air is inducted at room temp so:
# T1_ideal = ROOM_TEMP_K

# # Use polytropic relationships to find T2
# T2_ideal = T1_ideal * (V1_ideal / V2_ideal)**(n_comp - 1)

# # Use constant pressure assumption to find T3
```

```
# T3_ideal = T2_ideal * V3_ideal / V2_ideal

# Use CoolProp to find the Cp of air at T2
Cp_air_T2_ideal = PropsSI('Cpmass', 'T', T2_ideal, 'P', (P2_ideal * 1000), 'Air')
Cp_air_T2_ideal_kJ = Cp_air_T2_ideal / 1000

# Use CoolProp to find the Cp of air at T3
Cp_air_T3_ideal = PropsSI('Cpmass', 'T', T3_ideal, 'P', (P3_ideal * 1000), 'Air')
Cp_air_T3_ideal_kJ = Cp_air_T3_ideal / 1000

Cp_air_avg_T2_T3_kJ = (Cp_air_T2_ideal_kJ + Cp_air_T3_ideal_kJ) / 2

# Calculate heat input
Q_heat_input_kW = m_flow_rate_exh_kg_s * Cp_air_avg_T2_T3_kJ * (T3_ideal - T2_ideal) / 4

# Express heat input as a percentage of chemical fuel added by fuel.
fuel_energy_in_kW = (m_flow_rate_fuel_kg_s / 4) * DIESEL_LHV_MJ_KG * 1000
Q_heat_input_perc = Q_heat_input_kW / fuel_energy_in_kW * 100

# Plot complete set of data and the points identified
plt.figure(num=5, figsize=(8, 6))
plt.plot(vol_data, prs_data, label='Experimental Data')

point_size = 5

plt.plot(V1, P1, 'ro', markersize=point_size)
plt.plot(V1_ideal, P1_ideal, 'ko', markersize=point_size)

plt.plot(V2, P2, 'ro', markersize=point_size)
plt.plot(V2_ideal, P2_ideal, 'ko', markersize=point_size)

plt.plot(V3, P3, 'ro', markersize=point_size)
plt.plot(V3_ideal, P3_ideal, 'ko', markersize=point_size)

plt.plot(V4, P4, 'ro', markersize=point_size)
plt.plot(V4_ideal, P4_ideal, 'ko', markersize=point_size)

plt.plot(
    vol_data_ideal_comp, prs_data_ideal_comp, label='Fitted Air Standard Cycle',
    color='black', linestyle='--'
)
plt.plot(vol_data_ideal_pconst, prs_data_ideal_pconst, color='black', linestyle='--')
plt.plot(vol_data_ideal_exp, prs_data_ideal_exp, color='black', linestyle='--')
plt.plot(vol_data_ideal_vconst, prs_data_ideal_vconst, color='black', linestyle='--')

plt.axvline(x=min_vol, color='red', linestyle='--', linewidth=1)
plt.axvline(x=max_vol, color='red', linestyle='--', linewidth=1)

plt.title(f"PV Graph for Diesel Cycle at {engine_rpm} rpm", fontsize=16)
plt.xlabel('Volume [m^3]')
plt.ylabel('Pressure [kPa]')

plt.annotate(
    "(1)",
    (V1_ideal, P1_ideal),
    textcoords="offset points",
```

```
    xytext=(12, -3),
    ha='center',
    fontsize=10
)

plt.annotate(
    "(2)",
    (V2_ideal, P2_ideal),
    textcoords="offset points",
    xytext=(-12, -3),
    ha='center',
    fontsize=10
)

plt.annotate(
    "(3)",
    (V3_ideal, P3_ideal),
    textcoords="offset points",
    xytext=(12, -3),
    ha='center',
    fontsize=10
)

plt.annotate(
    "(4)",
    (V4_ideal, P4_ideal),
    textcoords="offset points",
    xytext=(12, -3),
    ha='center',
    fontsize=10
)

plt.grid(True)
plt.legend()

#plt.tight_layout()
plt.show()

# Display results
print("-----------------------------------------------------")
print(f"Measured engine rpm: {engine_rpm_actual}")
print("-----------------------------------------------------")
print("-----------------------------------------------------")
print(f"Polytropic coefficient of compression: {n_comp:.5f}")
print("-----------------------------------------------------")
print(f"Polytropic coefficient of expansion: {n_exp:.5f}")
print("-----------------------------------------------------")
print(f"Indicated work/cylinder: {work_i_kJ_cyl:.5f} kJ")
print("-----------------------------------------------------")
print(f"Indicated power: {power_i_kW:.5f} kW")
print("-----------------------------------------------------")
print(f"Mechanical efficiency: {eff_mech_perc:.5f} %")
print("-----------------------------------------------------")
print(f"Indicated mean effective pressure: {imep_MPa:.5f} MPa")
print("-----------------------------------------------------")
print(f"Cut-Off Ratio: {cut_off_ratio:.5f} ul")
```

```
print("-----------------------------------------------------")
print(f"Fuel energy in per cylinder: {fuel_energy_in_kW:.5f} kW")
print("-----------------------------------------------------")
print(f"Heat input per cylinder: {Q_heat_input_kW:.5f} kW")
print("-----------------------------------------------------")
print(f"Heat input as percentage of fuel energy: {Q_heat_input_perc:.5f} %")
print("-----------------------------------------------------")

# Hold console window open if script is run from windows explorer
if not hasattr(sys, 'ps1'):
    input("Press Enter to exit...")
```